

Exploring SHS: Jobs, Stages and Tasks

In the past few lectures, we have visited SHS (Spark History Server) multiple times to understand how our application is performing. Now in this lecture we are going to understand various aspects of SHS.

Jobs Tab

There are various info being displayed in Jobs Tab. Lets take a look at each of them to understand it in detail

Spark Jobs (?)

User: svc-dragon-test

Total Uptime: 50 s

Scheduling Mode: FIFO

Completed Jobs: 18

Completed Jobs (18)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id (Job Group) +	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
17	count at SparkUsabilityApp.scala:155 count at SparkUsabilityApp.scala:155	2024/11/18 09:42:13	20 ms	1/1 (1 skipped)	1/1 (9 skipped)
16	count at SparkUsabilityApp.scala:155 count at SparkUsabilityApp.scala:155	2024/11/18 09:42:13	0.3 s	1/1	9/9
15	count at SparkUsabilityApp.scala:152 count at SparkUsabilityApp.scala:152	2024/11/18 09:42:13	33 ms	1/1 (1 skipped)	1/1 (9 skipped)
14	count at SparkUsabilityApp.scala:152 count at SparkUsabilityApp.scala:152	2024/11/18 09:42:12	0.5 s	1/1	9/9
13	count at SparkUsabilityApp.scala:144 count at SparkUsabilityApp.scala:144	2024/11/18 09:42:12	37 ms	1/1 (1 skipped)	1/1 (9 skipped)
12	count at SparkUsabilityApp.scala:144 count at SparkUsabilityApp.scala:144	2024/11/18 09:42:11	0.4 s	1/1	9/9
11	count at SparkUsabilityApp.scala:141 count at SparkUsabilityApp.scala:141	2024/11/18 09:42:11	22 ms	1/1 (1 skipped)	1/1 (9 skipped)
10	count at SparkUsabilityApp.scala:141 count at SparkUsabilityApp.scala:141	2024/11/18 09:42:10	1 s	1/1	9/9
9	count at SparkUsabilityApp.scala:111 count at SparkUsabilityApp.scala:111	2024/11/18 09:42:09	88 ms	1/1 (1 skipped)	1/1 (1 skipped)
8	count at SparkUsabilityApp.scala:111 count at SparkUsabilityApp.scala:111	2024/11/18 09:42:09	0.3 s	1/1	1/1
7	parquet at SparkUsabilityApp.scala:110 parquet at SparkUsabilityApp.scala:110	2024/11/18 09:42:09	0.1 s	1/1	1/1

1. User

- **Description:** This column or section shows the username or ID of the user who submitted the job.
- **Purpose:** Helps track and attribute jobs to specific users in multi-tenant systems.
- **Significance:** Useful for auditing, monitoring user activity, or troubleshooting job issues.
 - Example: In a corporate environment, this might display individual developers or team service accounts.

2. Total Uptime

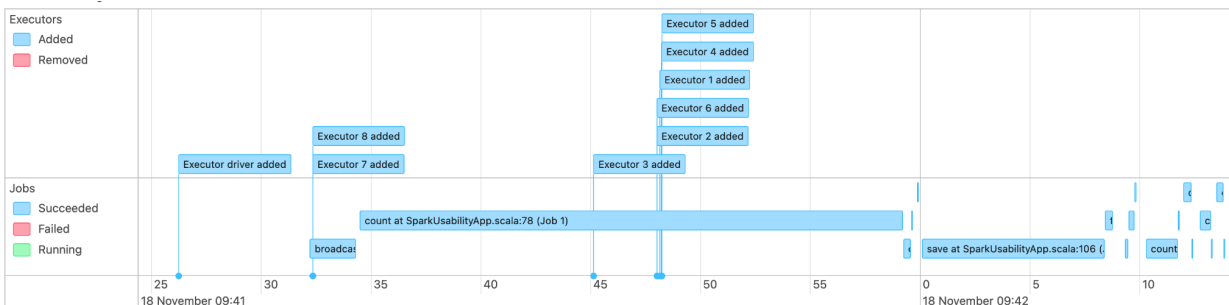
- **Description:** Represents the total duration for which the Spark application or job has been running.
- **Calculation:** Typically, it is the time elapsed since the job's start time until the current time (for active jobs) or until the job's completion time (for completed jobs).
- **Significance:** Helps monitor the efficiency of job execution:
 - Long uptimes could indicate inefficiencies, such as resource contention or suboptimal configurations.
 - Short uptimes may indicate optimized or lightweight jobs.

3. Scheduling Mode

- **Description:** Refers to the mode used for resource allocation and scheduling tasks within the job. Common scheduling modes in Spark include:
 - **FIFO (First In, First Out):** Jobs are executed in the order they are submitted.
 - **FAIR:** Resources are shared fairly across all running jobs.
 - **Purpose:** Defines how jobs are prioritized and how resources are distributed among them in multi-job scenarios.
 - **Significance:**
 - **FIFO** ensures predictable execution for jobs submitted earlier.
 - **FAIR** allows multiple jobs to run concurrently, preventing resource monopolization.
-

4. Completed Jobs

- **Description:** This tab or section lists all jobs that have finished execution.
- **Details Typically Included:**
 - **Job ID:** Unique identifier for the job.
 - **Status:** Whether the job succeeded or failed.
 - **Duration:** How long the job took to complete.
 - **Stages and Tasks:** Number of stages and tasks executed.
- **Purpose:** Provides a history of finished jobs for auditing, debugging, and performance tuning.
- **Significance:**
 - Allows you to investigate failed or inefficient jobs using logs or execution metrics.
 - Serves as a baseline for optimizing future workloads.



Event Timeline

The **Event Timeline** is a graphical representation of the lifecycle of a job or application in a distributed processing system (e.g., Spark). It shows events such as job submission, stage execution, and task completion as a timeline, often broken down into different phases or stages.

Components of the Event Timeline

1. **Job Execution Timeline:**
 - Tracks the start and end times for each job.
 - Useful for identifying bottlenecks at the job level.
 2. **Stage Timeline:**
 - Displays when stages of a job started and ended.
 - Includes stage dependencies and overlaps (if any).
 3. **Task Timeline:**
 - Shows execution times for individual tasks.
 - Highlights task distribution across executors, retries, or failures.
 4. **Executor/Resource Utilization:**
 - Indicates when resources like CPU or memory were active.
 - Helps identify resource contention or underutilization.
-

How the Event Timeline Helps in Debugging

The Event Timeline provides a visual and time-based breakdown of what happened during job execution, which can be invaluable for debugging. Here's how:

1. **Identify Bottlenecks:**
 - Locate stages or tasks that took significantly longer than others.
 - Example: If one stage's execution is much slower, it could indicate data skew or poor partitioning.
2. **Analyze Task Failures:**
 - Tasks that fail or retry frequently can be pinpointed on the timeline.
 - Helps determine if failures are caused by executor crashes, resource exhaustion, or bad input data.
3. **Understand Parallelism:**
 - Check if tasks are being run in parallel or sequentially.
 - Sequential task execution could suggest underutilization of cluster resources.
4. **Resource Utilization:**

- Correlate task execution times with resource usage patterns (e.g., memory spikes or I/O contention).
 - Detect issues like executor slowdowns or OutOfMemory errors.
-

Example: Debugging a Job Using the Event Timeline

Scenario: A Spark job is running slower than expected, taking 30 minutes to complete instead of the usual 10 minutes.

Steps for Debugging:

- 1. Inspect the Job Timeline:**
 - Observe if any particular job shows excessive runtime compared to others.
 - Suppose Job ID 5 takes 25 minutes, while others are under 5 minutes.
- 2. Analyze the Stage Timeline:**
 - Focus on the long-running stage within Job ID 5.
 - Identify Stage 3, which takes 22 minutes, as the bottleneck.
- 3. Drill Down into Task Execution:**
 - Check the Task Timeline for Stage 3.
 - Notice that 90% of tasks finish quickly, but 10% take an unusually long time.
- 4. Diagnose Data Skew:**
 - Realize that the long-running tasks are processing large partitions of data due to uneven data distribution.
 - The timeline highlights these tasks as outliers in duration.
- 5. Resolution:**
 - Adjust the partitioning logic in your Spark job to ensure more uniform data distribution.
 - Rerun the job and verify improvements via the Event Timeline.

Details of One Job

Details for Job 17

Status: SUCCEEDED

Submitted: 2024/11/18 09:42:13

Duration: 20 ms

Associated SQL Query: 6

Completed Stages: 1

Skipped Stages: 1

▶ Event Timeline

▶ DAG Visualization

▼ Completed Stages (1)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
38	count at SparkUsabilityApp.scala:155 +details	2024/11/18 09:42:13	17 ms	1/1				

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

▼ Skipped Stages (1)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
37	count at SparkUsabilityApp.scala:155 +details	Unknown	Unknown	0/9				

1. Status

- **Description:** Indicates the current state of the job.
 - **Running:** The job is actively executing.
 - **Succeeded:** The job completed successfully without errors.
 - **Failed:** The job encountered errors during execution.
 - **Skipped:** Certain stages or tasks were skipped due to optimizations or failures in dependent stages.
- **Significance:**
 - Helps quickly identify whether a job is progressing as expected.
 - Alerts users to failures or partial executions that may need attention.

2. Submitted

- **Description:** Shows the timestamp of when the job was submitted for execution.
- **Significance:**
 - Useful for tracking job submission times in production environments.
 - Helps correlate job submissions with other system activities, like resource allocation events or logs.

3. Duration

- **Description:** The total time the job took to complete (or the elapsed time for a running job).
 - Calculated as: $\text{Duration} = \text{Completion Time} - \text{Start Time}$
 - **Significance:**
 - Highlights performance issues.
 - Allows comparison with historical job runtimes to detect anomalies.
-

4. Associated SQL Query

- **Description:** Displays the SQL query (if any) that triggered the job.
 -
 - **Significance:**
 - Links jobs to the high-level logic, making it easier to debug query execution plans.
 - Helps identify how SQL statements are translated into jobs and stages.
-

5. Completed Stages

- **Description:** Lists the stages that have been successfully completed as part of the job execution.
 - Each stage represents a set of parallel tasks that Spark schedules and executes.
 - **Significance:**
 - Allows users to monitor progress in large, multi-stage jobs.
 - Helps in identifying bottlenecks if certain stages take significantly longer to complete.
-

6. DAG Visualization

- **Description:** Provides a **Directed Acyclic Graph (DAG)** representation of the job's execution plan.
 - Nodes represent stages.
 - Edges show dependencies between stages.
- **Significance:**
 - Visualizes how data flows through different stages.

- Helps in understanding execution logic and stage dependencies.
 - Useful for detecting inefficiencies, such as redundant shuffles or wide dependencies.
-

7. Skipped Stages

- **Description:** Lists the stages that were skipped during job execution.
 - **Why Stages Can Be Skipped:**
 - **Stage Caching:** If output from a stage is already cached, subsequent execution skips recalculating it.
 - **Stage Failures:** A downstream stage might be skipped if its dependent stages fail.
 - **Stage Sharing:** In Spark, multiple actions (e.g., `count()` and `collect()`) on the same RDD/DataFrame can share the same stage, skipping redundant executions.
 - **Result Reuse:** Optimizations like query result caching or reused shuffle outputs avoid repeating computations.
 - **Significance:**
 - Improves job performance by reusing previously computed results.
 - Reduces resource usage in cases where intermediate data is cached or reused.
-

Example of Skipped Stages

Scenario:

You run a Spark SQL query like:

```
SELECT COUNT(*) FROM events WHERE event_type = 'click';
```

This query is executed in two actions:

1. **Action 1:** `cache()` is applied to filter results where `event_type = 'click'`.
2. **Action 2:** `COUNT(*)` is performed on the cached result.

In this case:

- The stage responsible for filtering and caching is executed once.

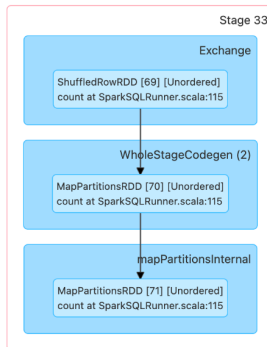
- When `COUNT(*)` is executed, the filtering stage is skipped because the result is reused from cache.

Stage Tab

Details for Stage 33 (Attempt 0)

Resource Profile Id: 0
 Total Time Across All Tasks: 9 ms
 Locality Level Summary: Process local: 1
 Associated Job Ids: 21

▼ DAG Visualization



In the **Stage Tab** of the **Spark History Server (SHS)**, several important metrics and features provide insights into the performance and execution details of each stage in a Spark application. Here's a breakdown of the terms you mentioned:

1. Total Time Across All Tasks

- This metric represents the cumulative time spent executing all tasks within a specific stage.
- **Calculation:**
$$\text{Total Time Across All Tasks} = \sum_{i=1}^N (\text{Finish Time}_i - \text{Start Time}_i)$$

$$\text{Total Time Across All Tasks} = \sum_{i=1}^N (\text{Finish Time}_i - \text{Start Time}_i)$$
 where N is the total number of tasks in the stage.
- It accounts for both **successful** and **failed** tasks.
- **Importance:**
 - Indicates the computational effort for the stage.
 - Helps identify bottlenecks (e.g., tasks taking unusually long).
 - Useful for comparing the performance of stages and diagnosing inefficiencies.

2. Locality

- **Locality Levels:**
 - Spark tries to execute tasks as close to the data location as possible (data locality).
 - Levels:
 - **PROCESS_LOCAL:** Task runs on the same node as the data.
 - **NODE_LOCAL:** Task runs on the same node but fetches data over the network.
 - **RACK_LOCAL:** Task runs on a different node in the same rack.
 - **ANY:** Task runs anywhere in the cluster.
- **Why It Matters:**
 - The closer the task is to the data, the lower the data transfer overhead.
 - High percentages of **PROCESS_LOCAL** tasks indicate efficient scheduling.
 - Poor locality (e.g., many **ANY** tasks) may signal data skew or storage issues.

3. Associated Job IDs

- Each stage is part of one or more Spark jobs, depending on dependencies.
- The **Stage Tab** shows the **Job ID(s)** linked to the stage under a dedicated column.
- **Purpose:**
 - Helps trace which job a stage belongs to, especially in multi-job applications.
 - Provides context for understanding the stage's role in the overall application.
 - You can click the Job ID to navigate to the **Jobs Tab** for a broader view.

4. DAG Visualization

- Spark generates a **Directed Acyclic Graph (DAG)** to represent the sequence of computations (transformations and actions).
- **In the Stage Tab:**
 - You can view the **DAG visualization** for the selected stage.

- Highlights the dependencies between RDDs, stages, and their parent-child relationships.
- **Interactivity:**
 - Hovering over a stage node shows its details (e.g., task count, shuffle dependencies).
 - Helps identify bottlenecks in the execution flow, such as expensive shuffles.
- **Usefulness:**
 - Provides a visual summary of the logical and physical execution plan.
 - Assists in debugging performance issues like skewed data or redundant shuffles.

5. Event Timeline

- A graphical representation of the timing information for the stage and its tasks.
- **Components:**
 - **X-Axis:** Time elapsed (milliseconds or seconds).
 - **Bars/Lines:** Each bar represents a task's execution time.
 - Different colors indicate task states (e.g., running, successful, failed, etc.).
- **Insights Provided:**
 - Distribution of task durations (e.g., if some tasks are much slower than others, indicating skew).
 - Gaps or overlaps in task execution, which can suggest issues like resource contention or executor failures.
 - Helps correlate task delays with locality issues or shuffle overhead.

Aggregated Metrics by Executor

Show entries Search:

Executor ID	Logs	Address	Task Time	Total Tasks	Failed Tasks	Killed Tasks	Succeeded Tasks	Excluded
1	stdout stderr	phx6-3sh.prod.uber.internal:40629	45.0 ms	1	0	0	1	false

Showing 1 to 1 of 1 entries Previous **1** Next

Tasks (1)

Show entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Errors
0	21	0	SUCCESS	PROCESS_LOCAL	1	phx6-3sh.prod.uber.internal	stdout stderr	2024-11-21 12:47:49	9.0 ms		

6. Aggregated Metrics by Executors

This section displays metrics aggregated at the executor level, showing how each executor performed during the stage.

Key Metrics:

1. **Executor ID:**
 - Identifies the executor that ran tasks for the stage.
 - Clicking the ID often provides more detailed executor information.
2. **Task Time:**
 - Total time spent by all tasks executed on the executor.
 - **Interpretation:**
 - High task times may indicate uneven task distribution or slower hardware for that executor.
3. **Shuffle Read/Write:**
 - **Shuffle Read:**
 - Amount of data read by tasks on the executor during shuffles.
 - High values might signal a large shuffle stage or data skew.
 - **Shuffle Write:**
 - Data written to shuffle storage by tasks on this executor.
 - Large shuffle write values suggest expensive operations (e.g., wide transformations).
4. **Input Bytes:**
 - Amount of input data read by the executor, often from HDFS or other storage systems.
 - Disproportionate values could indicate partitioning issues or an unbalanced workload.
5. **Output Bytes:**
 - Data written to the next stage or output storage by the executor.
6. **GC Time:**
 - Time spent in garbage collection (GC) by the executor.
 - High GC times can indicate memory pressure or inefficient memory usage.
7. **Peak Memory Usage:**
 - Maximum memory used by the executor during task execution.
 - Useful for diagnosing out-of-memory (OOM) errors or memory-intensive workloads.

7. Aggregated Metrics by Tasks

This section provides task-level aggregation metrics to show how the tasks performed within the stage.

Key Metrics:

1. **Task ID:**
 - Unique identifier for each task within the stage.
2. **Duration:**
 - Total time taken by the task, including:
 - Execution time
 - Time spent waiting for resources (e.g., scheduler delays)
 - **Interpretation:**
 - Longer task durations may indicate:
 - Data skew
 - Stragglers
 - Network delays (for shuffle-heavy stages)
3. **Task Deserialization Time:**
 - Time spent deserializing the task on the executor.
 - High deserialization times might indicate serialization inefficiencies or large task payloads.
4. **GC Time:**
 - Time spent by the JVM on garbage collection during the task.
 - High GC times can indicate excessive object creation or memory mismanagement.
5. **Shuffle Read/Write:**
 - **Shuffle Read:**
 - Amount of shuffle data read by the task.
 - High shuffle read values can suggest uneven partitioning.
 - **Shuffle Write:**
 - Amount of data written to shuffle storage by the task.
6. **Input/Output Bytes:**
 - **Input Bytes:**
 - Data read by the task from storage.
 - **Output Bytes:**
 - Data produced by the task, written to downstream stages or final output.
7. **Locality Level:**
 - Indicates the data locality for the task (e.g., process-local, node-local, rack-local, or any).
 - Tasks with poor locality may experience delays due to data transfer.
8. **Task Result Fetch Time:**

- Time taken to fetch the results of the task from the executor back to the driver.
-

How to Use These Metrics:

Identifying Executor Bottlenecks:

- **Uneven Task Distribution:**
 - Check if some executors have significantly higher task times or shuffle read/write compared to others.
 - Resolve this by repartitioning or optimizing data skew.
- **Memory Issues:**
 - Look for high GC time or peak memory usage on specific executors.
 - Tune executor memory settings to reduce memory pressure.

Diagnosing Task Stragglers:

- **Outliers in Task Duration:**
 - Identify tasks that take significantly longer than others.
 - Stragglers can result from data skew, poor locality, or resource contention.
- **Shuffle Performance:**
 - High shuffle read/write values for specific tasks or executors may indicate expensive shuffles due to wide transformations.
- **Deserialization Delays:**
 - High task deserialization times may suggest inefficiencies in serialization or large closures.